

Image Assignment ADNE: Pretrained Models

By Guillermo Velasco, David Fernández-Madera y Rafael de Bloteau

In this project, we aim to classify various eye diseases using a labeled image dataset. To accomplish this, we leverage the power of transfer learning by fine-tuning four state-of-the-art convolutional neural networks (CNNs) pretrained on ImageNet. The selected models vary in complexity and architecture, allowing us to compare their performance across different levels of representational power and computational cost.

The four models used are, from most complex to least complex:

- RegNetY-128GF
- ResNeXt101_64x4d
- ResNeXt50_32x4d
- ResNet50

These models have been chosen for their proven effectiveness in image classification tasks

Hypothesis: ¶

Given their complexity and design we hypothesize that:

- RegNetY-128GF will outperform all other models, especially in capturing fine-grained patterns in eye disease images due to its depth and architectural sophistication.
- ResNet50 is expected to have the lowest performance, as it lacks the enhanced feature representation capabilities of the ResNeXt and RegNet architectures.
- ResNeXt101_64x4d and ResNeXt50_32x4d are expected to fall in between, with the deeper ResNeXt101 likely outperforming its smaller counterpart.

Import Libraries

```
In [ ]: from cnn import CNN, load_data
import torchvision
import torch
import torch.nn as nn
import wandb
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
import pandas as pd
import numpy as np
import os
import torch
import torch.nn as nn
import torchvision
import matplotlib.pyplot as plt

import os
```

PyTorch Device Setup

```
In [ ]: # Configurar dispositivo
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"✅ Running on: {device}")
if device.type == "cuda":
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

✅ Running on: cuda
GPU: NVIDIA GeForce RTX 2060

Train and Validation Paths

```
In [ ]: # Paths
train_dir = "C:\\Users\\guill\\Documents\\master\\ImagenAdne\\PracticaImagen\\dataset\\train"
valid_dir = "C:\\Users\\guill\\Documents\\master\\ImagenAdne\\PracticaImagen\\dataset\\val"
```

Load Data and set of Hyperparameters

```
In [ ]: # One set of hyperparameters
batch_size = 32
img_size = 224
learning_rate = 0.001
epochs = 40
unfrozen_layer_names = ['layer4', 'fc']

# Load data
train_loader, valid_loader, num_classes = load_data(train_dir, valid_dir, batch_size, img_size)
```

Models

1. resnext101_64x4d

ResNeXt101_64x4d is a deep CNN that enhances ResNet by using multiple parallel paths (called cardinality) in each block. With 101 layers and a 64x4d configuration, it captures complex features efficiently, making it powerful for tasks like medical image classification.

1.1 resnext101_64x4d code


```

In [ ]: # Load model
base_model = torchvision.models.resnext101_64x4d(weights='DEFAULT')
model = CNN(base_model, num_classes)
model.to(device)

# Freeze everything
for param in model.parameters():
    param.requires_grad = False

# Unfreeze selected layers
print("✅ Unfreezing layers:", unfrozen_layer_names)
for name, module in base_model.named_children():
    if name in unfrozen_layer_names:
        for param in module.parameters():
            param.requires_grad = True

# Ensure classifier (fc) head is unfreezed from custom model
for param in model.fc.parameters():
    param.requires_grad = True

# Print trainable param count
trainable_params = list(filter(lambda p: p.requires_grad, model.parameters()))
print(f"🔍 Trainable parameters: {sum(p.numel() for p in trainable_params)}")

if not trainable_params:
    raise RuntimeError("❌ No trainable parameters found. Check layer unfreezing logic.")

# Optimizer and Loss
optimizer = torch.optim.Adam(trainable_params, lr=learning_rate)
criterion = nn.CrossEntropyLoss()

# Train
history = model.train_model(train_loader, valid_loader, optimizer, criterion, epochs=epochs, device=device)

# Save model
model_filename = f"resnext101_64x4d_model_batchsize{batch_size}_imgsize{img_size}_learningrate{learning_rate}_ep
model_path = os.path.join(os.getcwd(), model_filename)
torch.save(model.state_dict(), model_path)

# === Save Plots === #
plot_folder = os.path.join("plots", os.path.splitext(model_filename)[0])
os.makedirs(plot_folder, exist_ok=True)

```

```
def plot_metric(metric_name, train_values, val_values):
    plt.figure()
    plt.plot(range(1, epochs + 1), train_values, label='Train')
    plt.plot(range(1, epochs + 1), val_values, label='Validation')
    plt.xlabel("Epoch")
    plt.ylabel(metric_name.capitalize())
    plt.title(f"{metric_name.capitalize()} over Epochs")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(plot_folder, f"{metric_name}.png"))
    plt.close()

# Plot and save all relevant metrics
plot_metric("loss", history["train_loss"], history["valid_loss"])
plot_metric("accuracy", history["train_accuracy"], history["valid_accuracy"])

# === Evaluate and Save Confusion Matrix === #
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in valid_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, preds = torch.max(outputs, 1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Compute confusion matrix
cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues', xticks_rotation='vertical')

# Save confusion matrix plot
plt.title("Confusion Matrix")
plt.savefig(os.path.join(plot_folder, "confusion_matrix.png"))
plt.close()

# Save classification report
report = classification_report(all_labels, all_preds, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.to_csv(os.path.join(plot_folder, "classification_report.csv"))
```

```
# Optional: also save raw confusion matrix  
cm_df = pd.DataFrame(cm)  
cm_df.to_csv(os.path.join(plot_folder, "confusion_matrix_raw.csv"), index=False)
```

✓ Unfreezing layers: ['layer4', 'fc']

🔍 Trainable parameters: 29078532

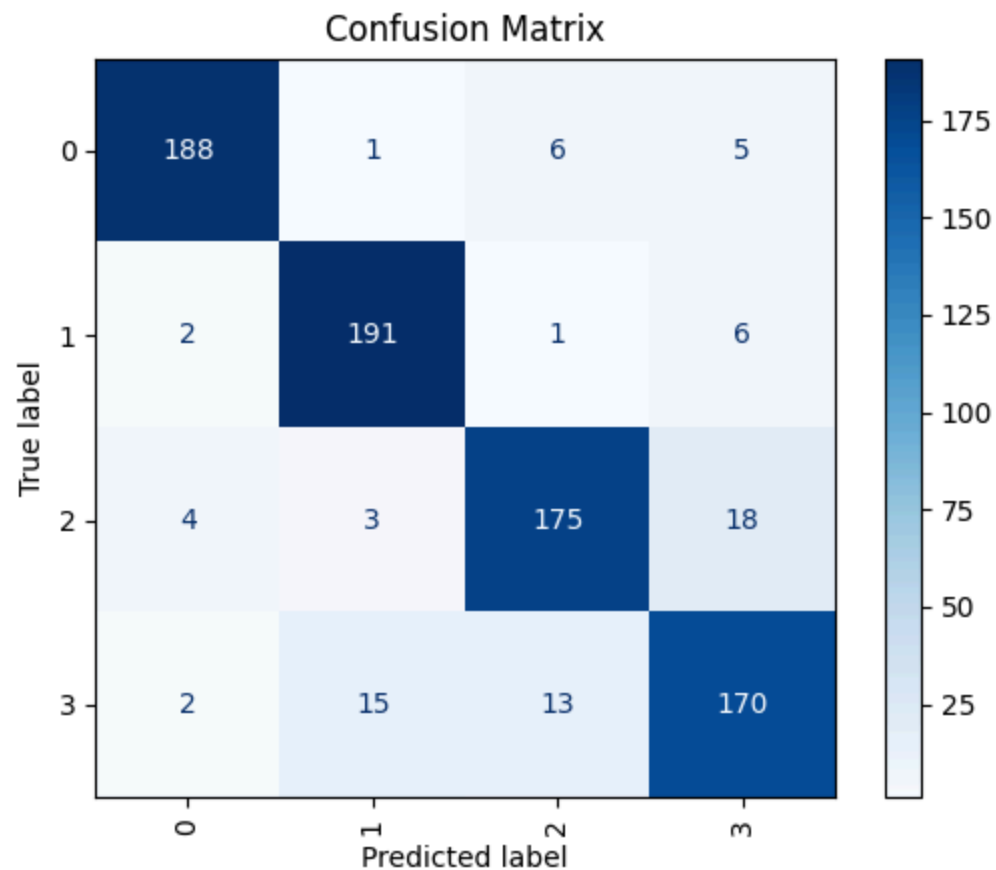
```
Epoch 1/40 - Train Loss: 0.7321, Train Acc: 0.7078
Epoch 1/40 - Val Loss: 0.7840, Val Acc: 0.7525
Epoch 2/40 - Train Loss: 0.4969, Train Acc: 0.8147
Epoch 2/40 - Val Loss: 0.4133, Val Acc: 0.8375
Epoch 3/40 - Train Loss: 0.4691, Train Acc: 0.8214
Epoch 3/40 - Val Loss: 0.4739, Val Acc: 0.8425
Epoch 4/40 - Train Loss: 0.3883, Train Acc: 0.8528
Epoch 4/40 - Val Loss: 0.4139, Val Acc: 0.8550
Epoch 5/40 - Train Loss: 0.3661, Train Acc: 0.8631
Epoch 5/40 - Val Loss: 0.4220, Val Acc: 0.8575
Epoch 6/40 - Train Loss: 0.3448, Train Acc: 0.8701
Epoch 6/40 - Val Loss: 0.6283, Val Acc: 0.8237
Epoch 7/40 - Train Loss: 0.3005, Train Acc: 0.8821
Epoch 7/40 - Val Loss: 0.5623, Val Acc: 0.8087
Epoch 8/40 - Train Loss: 0.2935, Train Acc: 0.8871
Epoch 8/40 - Val Loss: 0.5972, Val Acc: 0.7913
Epoch 9/40 - Train Loss: 0.2848, Train Acc: 0.8995
Epoch 9/40 - Val Loss: 0.3942, Val Acc: 0.8612
Epoch 10/40 - Train Loss: 0.2910, Train Acc: 0.8952
Epoch 10/40 - Val Loss: 0.3610, Val Acc: 0.8788
Epoch 11/40 - Train Loss: 0.2646, Train Acc: 0.8925
Epoch 11/40 - Val Loss: 0.4585, Val Acc: 0.8337
Epoch 12/40 - Train Loss: 0.2761, Train Acc: 0.8985
Epoch 12/40 - Val Loss: 0.4535, Val Acc: 0.8475
Epoch 13/40 - Train Loss: 0.2270, Train Acc: 0.9132
Epoch 13/40 - Val Loss: 0.4404, Val Acc: 0.8413
Epoch 14/40 - Train Loss: 0.2258, Train Acc: 0.9078
Epoch 14/40 - Val Loss: 0.3421, Val Acc: 0.8825
Epoch 15/40 - Train Loss: 0.2286, Train Acc: 0.9189
Epoch 15/40 - Val Loss: 0.4125, Val Acc: 0.8475
Epoch 16/40 - Train Loss: 0.2441, Train Acc: 0.9095
Epoch 16/40 - Val Loss: 0.3774, Val Acc: 0.8775
Epoch 17/40 - Train Loss: 0.2058, Train Acc: 0.9199
Epoch 17/40 - Val Loss: 0.3961, Val Acc: 0.8638
Epoch 18/40 - Train Loss: 0.2053, Train Acc: 0.9215
Epoch 18/40 - Val Loss: 0.3621, Val Acc: 0.8825
Epoch 19/40 - Train Loss: 0.1841, Train Acc: 0.9346
Epoch 19/40 - Val Loss: 0.3489, Val Acc: 0.8875
Epoch 20/40 - Train Loss: 0.1955, Train Acc: 0.9309
Epoch 20/40 - Val Loss: 0.3595, Val Acc: 0.8925
Epoch 21/40 - Train Loss: 0.1756, Train Acc: 0.9339
```


Epoch 21/40 - Val Loss: 0.3793, Val Acc: 0.8850
Epoch 22/40 - Train Loss: 0.2037, Train Acc: 0.9229
Epoch 22/40 - Val Loss: 0.5102, Val Acc: 0.8612
Epoch 23/40 - Train Loss: 0.2052, Train Acc: 0.9229
Epoch 23/40 - Val Loss: 0.3298, Val Acc: 0.8925
Epoch 24/40 - Train Loss: 0.1906, Train Acc: 0.9302
Epoch 24/40 - Val Loss: 0.4563, Val Acc: 0.8612
Epoch 25/40 - Train Loss: 0.1765, Train Acc: 0.9352
Epoch 25/40 - Val Loss: 0.4793, Val Acc: 0.8738
Epoch 26/40 - Train Loss: 0.1755, Train Acc: 0.9346
Epoch 26/40 - Val Loss: 0.4914, Val Acc: 0.8588
Epoch 27/40 - Train Loss: 0.1589, Train Acc: 0.9362
Epoch 27/40 - Val Loss: 0.4279, Val Acc: 0.8962
Epoch 28/40 - Train Loss: 0.1714, Train Acc: 0.9352
Epoch 28/40 - Val Loss: 0.3847, Val Acc: 0.8812
Epoch 29/40 - Train Loss: 0.1593, Train Acc: 0.9426
Epoch 29/40 - Val Loss: 0.3657, Val Acc: 0.8875
Epoch 30/40 - Train Loss: 0.1659, Train Acc: 0.9392
Epoch 30/40 - Val Loss: 0.3535, Val Acc: 0.8925
Epoch 31/40 - Train Loss: 0.1782, Train Acc: 0.9336
Epoch 31/40 - Val Loss: 0.5953, Val Acc: 0.8375
Epoch 32/40 - Train Loss: 0.1500, Train Acc: 0.9442
Epoch 32/40 - Val Loss: 0.3742, Val Acc: 0.8712
Epoch 33/40 - Train Loss: 0.1745, Train Acc: 0.9379
Epoch 33/40 - Val Loss: 0.4153, Val Acc: 0.8875
Epoch 34/40 - Train Loss: 0.1497, Train Acc: 0.9466
Epoch 34/40 - Val Loss: 0.3456, Val Acc: 0.9050
Epoch 35/40 - Train Loss: 0.1735, Train Acc: 0.9382
Epoch 35/40 - Val Loss: 0.3814, Val Acc: 0.8912
Epoch 36/40 - Train Loss: 0.1507, Train Acc: 0.9442
Epoch 36/40 - Val Loss: 0.3519, Val Acc: 0.9050
Epoch 37/40 - Train Loss: 0.1522, Train Acc: 0.9462
Epoch 37/40 - Val Loss: 0.4259, Val Acc: 0.8850
Epoch 38/40 - Train Loss: 0.1469, Train Acc: 0.9476
Epoch 38/40 - Val Loss: 0.6806, Val Acc: 0.8638
Epoch 39/40 - Train Loss: 0.1377, Train Acc: 0.9469
Epoch 39/40 - Val Loss: 0.3591, Val Acc: 0.8962
Epoch 40/40 - Train Loss: 0.1412, Train Acc: 0.9472
Epoch 40/40 - Val Loss: 0.4724, Val Acc: 0.8925

1.2 resnext101_64x4d Results

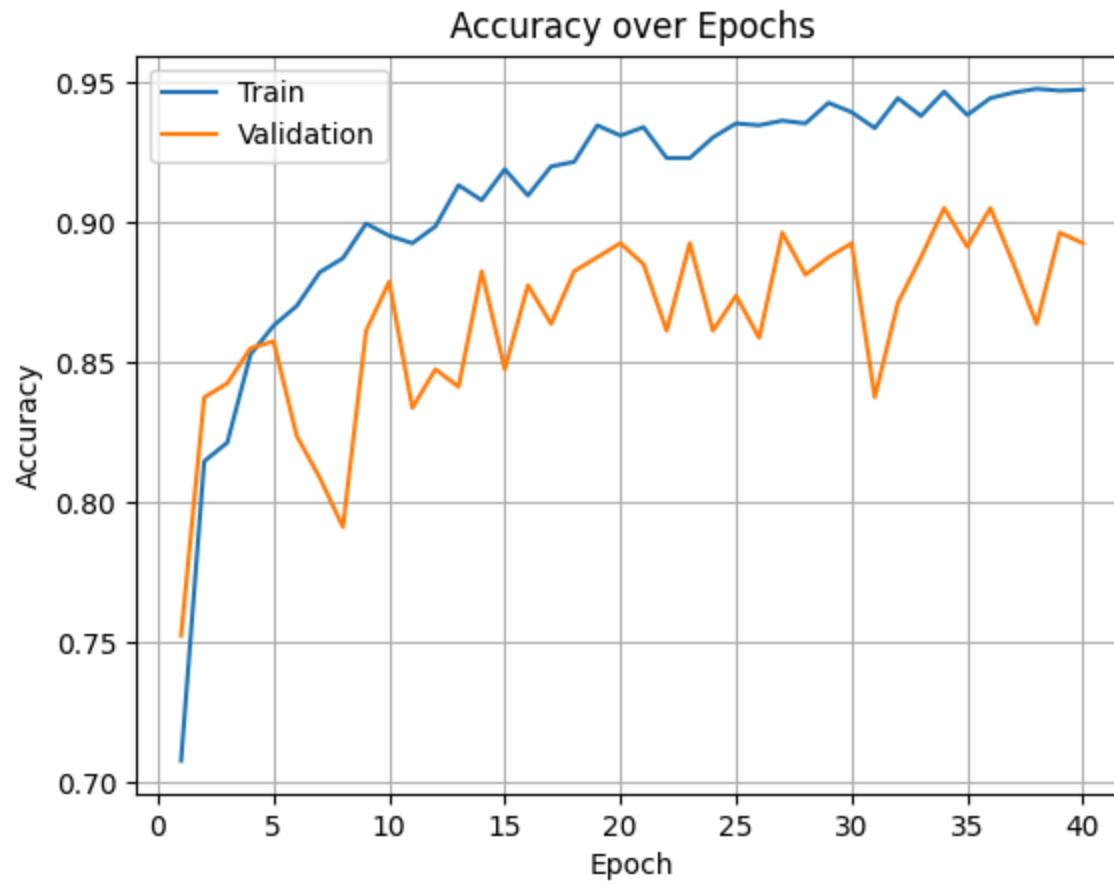
1.2.1 Confusion Matrix:

The confusion matrix indicates that the model is classifying most images correctly across all four classes. The majority of predictions lie along the diagonal, suggesting a strong correspondence between the predicted and true labels. Class 0 and Class 1 show particularly high levels of precision, with minimal misclassifications. While Class 2 and Class 3 are also predicted with good accuracy, there is slightly more confusion between them.



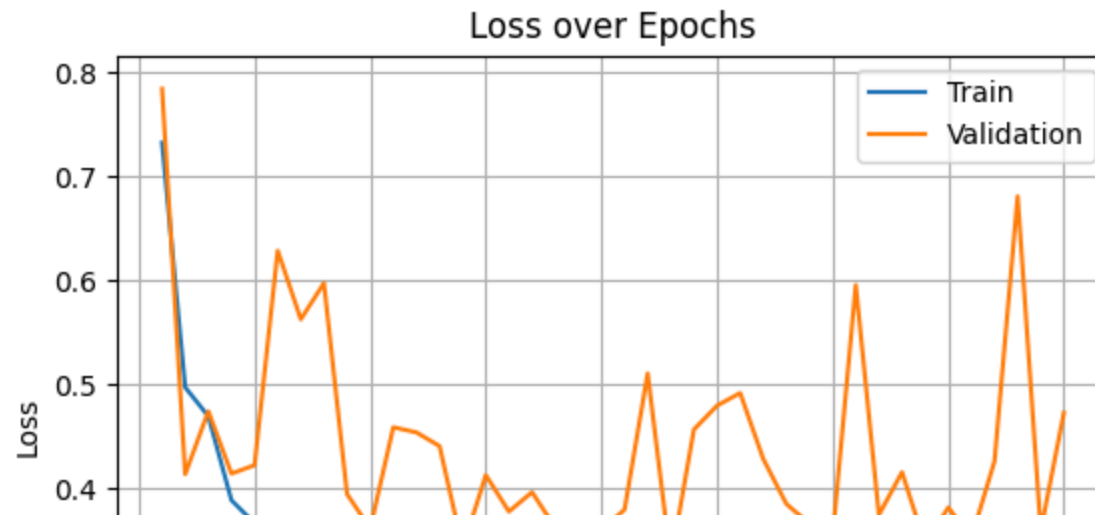
1.2.2 Accuracy over epoch

In the accuracy plot, training accuracy shows a steady upward trend, reaching approximately 95% by the end of training. Validation accuracy also improves over time, though it exhibits some fluctuation, stabilizing between 88% and 91%. The validation accuracy remains relatively strong, indicating that the model maintains good generalization to unseen data.



1.2.2 Loss over epoch

The loss curve supports these observations. Training loss decreases consistently, demonstrating that the model continues to learn effectively throughout the epochs. The validation loss, on the other hand, displays noticeable fluctuations and occasional spikes.



2. regnet_y_128gf

RegNetY-128GF is a large and powerful convolutional neural network designed for high performance. It uses a regular, scalable architecture and includes Squeeze-and-Excitation (SE) blocks to enhance feature learning. With its high capacity, it's well-suited for complex tasks like eye disease classification.

2.1 regnet_y_128gf code


```

In [ ]: # Load model
base_model = torchvision.models.regnet_y_128gf(weights='DEFAULT')
model = CNN(base_model, num_classes)
model.to(device)

# Freeze everything
for param in model.parameters():
    param.requires_grad = False

# Unfreeze selected layers
print("✅ Unfreezing layers:", unfrozen_layer_names)
for name, module in base_model.named_children():
    if name in unfrozen_layer_names:
        for param in module.parameters():
            param.requires_grad = True

# Ensure classifier (fc) head is unfreezed from custom model
for param in model.fc.parameters():
    param.requires_grad = True

# Print trainable param count
trainable_params = list(filter(lambda p: p.requires_grad, model.parameters()))
print(f"🔍 Trainable parameters: {sum(p.numel() for p in trainable_params)}")

if not trainable_params:
    raise RuntimeError("❌ No trainable parameters found. Check layer unfreezing logic.")

# Optimizer and Loss
optimizer = torch.optim.Adam(trainable_params, lr=learning_rate)
criterion = nn.CrossEntropyLoss()

# Train
history = model.train_model(train_loader, valid_loader, optimizer, criterion, epochs=epochs, device=device)

# Save model
model_filename = f"regnet_y_128gf_model_batchsize{batch_size}_imgsize{img_size}_learningrate{learning_rate}_epoch{epoch}"
model_path = os.path.join(os.getcwd(), model_filename)
torch.save(model.state_dict(), model_path)

# === Save Plots === #
plot_folder = os.path.join("plots", os.path.splitext(model_filename)[0])
os.makedirs(plot_folder, exist_ok=True)

```

```
def plot_metric(metric_name, train_values, val_values):
    plt.figure()
    plt.plot(range(1, epochs + 1), train_values, label='Train')
    plt.plot(range(1, epochs + 1), val_values, label='Validation')
    plt.xlabel("Epoch")
    plt.ylabel(metric_name.capitalize())
    plt.title(f"{metric_name.capitalize()} over Epochs")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(plot_folder, f"{metric_name}.png"))
    plt.close()

# Plot and save all relevant metrics
plot_metric("loss", history["train_loss"], history["valid_loss"])
plot_metric("accuracy", history["train_accuracy"], history["valid_accuracy"])

# === Evaluate and Save Confusion Matrix === #
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in valid_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, preds = torch.max(outputs, 1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Compute confusion matrix
cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues', xticks_rotation='vertical')

# Save confusion matrix plot
plt.title("Confusion Matrix")
plt.savefig(os.path.join(plot_folder, "confusion_matrix.png"))
plt.close()

# Save classification report
report = classification_report(all_labels, all_preds, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.to_csv(os.path.join(plot_folder, "classification_report.csv"))
```

```
# Optional: also save raw confusion matrix
cm_df = pd.DataFrame(cm)
cm_df.to_csv(os.path.join(plot_folder, "confusion_matrix_raw.csv"), index=False)
```

✅ Unfreezing layers: ['layer4', 'fc']

🔍 Trainable parameters: 7574532

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:43: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
```

```
    scaler = GradScaler()
```

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:61: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with autocast():
```

Epoch 1/40 - Train Loss: 1.2660, Train Acc: 0.5927

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:86: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with autocast():
```

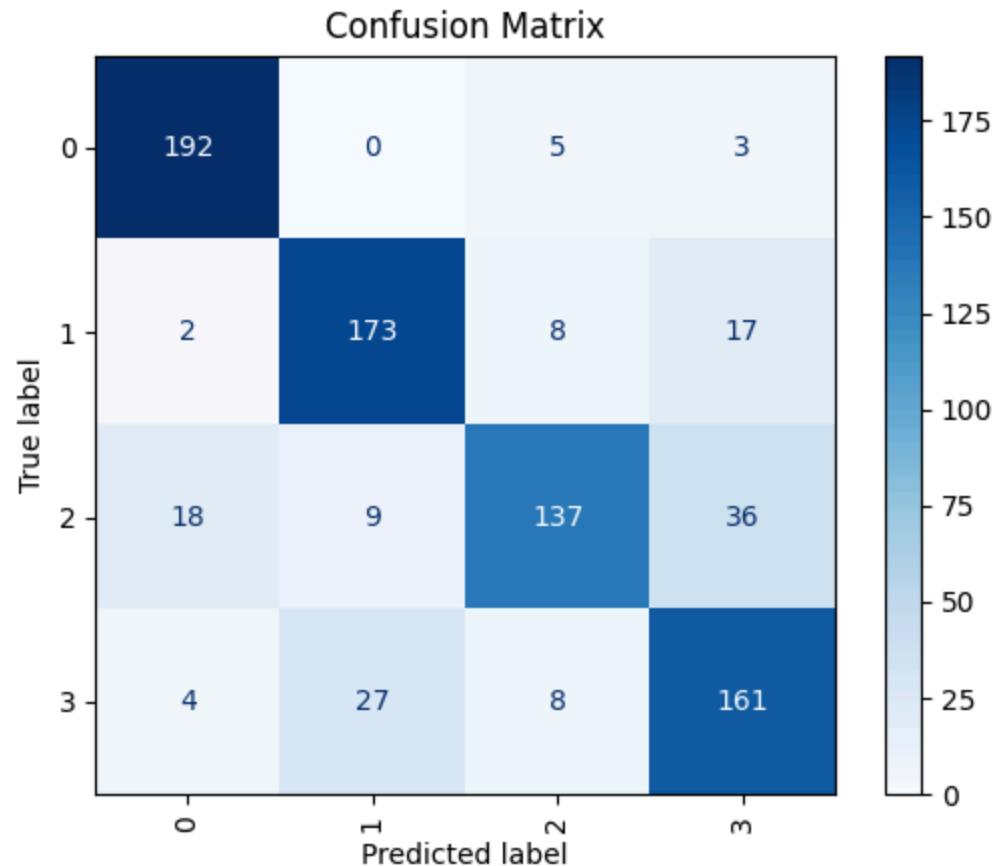

Epoch 1/40 - Val Loss: 0.8050, Val Acc: 0.6525
Epoch 2/40 - Train Loss: 0.8108, Train Acc: 0.6518
Epoch 2/40 - Val Loss: 0.7080, Val Acc: 0.6987
Epoch 3/40 - Train Loss: 0.7306, Train Acc: 0.7092
Epoch 3/40 - Val Loss: 0.8177, Val Acc: 0.6538
Epoch 4/40 - Train Loss: 0.6633, Train Acc: 0.7429
Epoch 4/40 - Val Loss: 0.6635, Val Acc: 0.7075
Epoch 5/40 - Train Loss: 0.6730, Train Acc: 0.7215
Epoch 5/40 - Val Loss: 0.6890, Val Acc: 0.7225
Epoch 6/40 - Train Loss: 0.6708, Train Acc: 0.7289
Epoch 6/40 - Val Loss: 0.5684, Val Acc: 0.7688
Epoch 7/40 - Train Loss: 0.6635, Train Acc: 0.7372
Epoch 7/40 - Val Loss: 0.6935, Val Acc: 0.7188
Epoch 8/40 - Train Loss: 0.6420, Train Acc: 0.7396
Epoch 8/40 - Val Loss: 0.6274, Val Acc: 0.7425
Epoch 9/40 - Train Loss: 0.6254, Train Acc: 0.7422
Epoch 9/40 - Val Loss: 0.6207, Val Acc: 0.7425
Epoch 10/40 - Train Loss: 0.6182, Train Acc: 0.7503
Epoch 10/40 - Val Loss: 0.7104, Val Acc: 0.7175
Epoch 11/40 - Train Loss: 0.6097, Train Acc: 0.7516
Epoch 11/40 - Val Loss: 0.5522, Val Acc: 0.7825
Epoch 12/40 - Train Loss: 0.6107, Train Acc: 0.7546
Epoch 12/40 - Val Loss: 0.5696, Val Acc: 0.7812
Epoch 13/40 - Train Loss: 0.5949, Train Acc: 0.7609
Epoch 13/40 - Val Loss: 0.5351, Val Acc: 0.7887
Epoch 14/40 - Train Loss: 0.5638, Train Acc: 0.7716
Epoch 14/40 - Val Loss: 0.5104, Val Acc: 0.8000
Epoch 15/40 - Train Loss: 0.5804, Train Acc: 0.7649
Epoch 15/40 - Val Loss: 0.6207, Val Acc: 0.7412
Epoch 16/40 - Train Loss: 0.5925, Train Acc: 0.7523
Epoch 16/40 - Val Loss: 0.5890, Val Acc: 0.7612
Epoch 17/40 - Train Loss: 0.5689, Train Acc: 0.7763
Epoch 17/40 - Val Loss: 0.5957, Val Acc: 0.7650
Epoch 18/40 - Train Loss: 0.5814, Train Acc: 0.7599
Epoch 18/40 - Val Loss: 0.6788, Val Acc: 0.7388
Epoch 19/40 - Train Loss: 0.5514, Train Acc: 0.7723
Epoch 19/40 - Val Loss: 0.5673, Val Acc: 0.7788
Epoch 20/40 - Train Loss: 0.5478, Train Acc: 0.7863
Epoch 20/40 - Val Loss: 0.5908, Val Acc: 0.7688
Epoch 21/40 - Train Loss: 0.5593, Train Acc: 0.7743
Epoch 21/40 - Val Loss: 0.5179, Val Acc: 0.8025
Epoch 22/40 - Train Loss: 0.5484, Train Acc: 0.7806
Epoch 22/40 - Val Loss: 0.4785, Val Acc: 0.8187

Epoch 23/40 - Train Loss: 0.5446, Train Acc: 0.7833
Epoch 23/40 - Val Loss: 0.4467, Val Acc: 0.8287
Epoch 24/40 - Train Loss: 0.5401, Train Acc: 0.7856
Epoch 24/40 - Val Loss: 0.4903, Val Acc: 0.8037
Epoch 25/40 - Train Loss: 0.5277, Train Acc: 0.7886
Epoch 25/40 - Val Loss: 0.4695, Val Acc: 0.8125
Epoch 26/40 - Train Loss: 0.5363, Train Acc: 0.7923
Epoch 26/40 - Val Loss: 0.5514, Val Acc: 0.7812
Epoch 27/40 - Train Loss: 0.5053, Train Acc: 0.8020
Epoch 27/40 - Val Loss: 0.4535, Val Acc: 0.8175
Epoch 28/40 - Train Loss: 0.5162, Train Acc: 0.7957
Epoch 28/40 - Val Loss: 0.5232, Val Acc: 0.7850
Epoch 29/40 - Train Loss: 0.5117, Train Acc: 0.8007
Epoch 29/40 - Val Loss: 0.5182, Val Acc: 0.8000
Epoch 30/40 - Train Loss: 0.5191, Train Acc: 0.7866
Epoch 30/40 - Val Loss: 0.5415, Val Acc: 0.7887
Epoch 31/40 - Train Loss: 0.5131, Train Acc: 0.8073
Epoch 31/40 - Val Loss: 0.5169, Val Acc: 0.8013
Epoch 32/40 - Train Loss: 0.5215, Train Acc: 0.7856
Epoch 32/40 - Val Loss: 0.4726, Val Acc: 0.8025
Epoch 33/40 - Train Loss: 0.5090, Train Acc: 0.7917
Epoch 33/40 - Val Loss: 0.4927, Val Acc: 0.8150
Epoch 34/40 - Train Loss: 0.5018, Train Acc: 0.7957
Epoch 34/40 - Val Loss: 0.5172, Val Acc: 0.8087
Epoch 35/40 - Train Loss: 0.4842, Train Acc: 0.8067
Epoch 35/40 - Val Loss: 0.4630, Val Acc: 0.8225
Epoch 36/40 - Train Loss: 0.5030, Train Acc: 0.7957
Epoch 36/40 - Val Loss: 0.4355, Val Acc: 0.8287
Epoch 37/40 - Train Loss: 0.4990, Train Acc: 0.8104
Epoch 37/40 - Val Loss: 0.5216, Val Acc: 0.8125
Epoch 38/40 - Train Loss: 0.4851, Train Acc: 0.8080
Epoch 38/40 - Val Loss: 0.5202, Val Acc: 0.8013
Epoch 39/40 - Train Loss: 0.4941, Train Acc: 0.8033
Epoch 39/40 - Val Loss: 0.4960, Val Acc: 0.8063
Epoch 40/40 - Train Loss: 0.5113, Train Acc: 0.7876
Epoch 40/40 - Val Loss: 0.5138, Val Acc: 0.8025

2.2 regnet_y_128gf Results

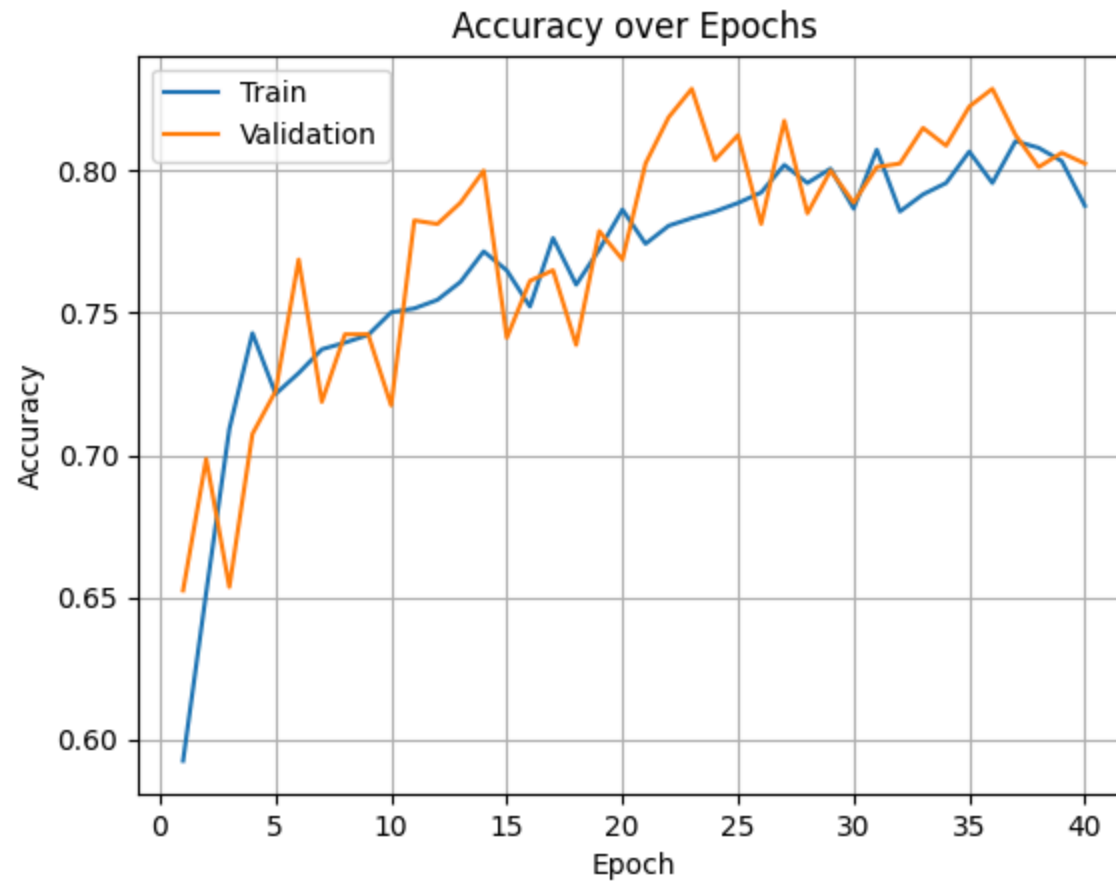
2.2.1 Confusion Matrix:

The confusion matrix reveals that the model performs well overall, particularly in correctly identifying Class 0, which shows almost perfect classification. However, there is noticeable confusion between Class 2 and Class 3, as well as between Class 1 and Class 3. Misclassifications are more frequent in these cases, suggesting that the model finds it more challenging to distinguish between these visually similar classes.



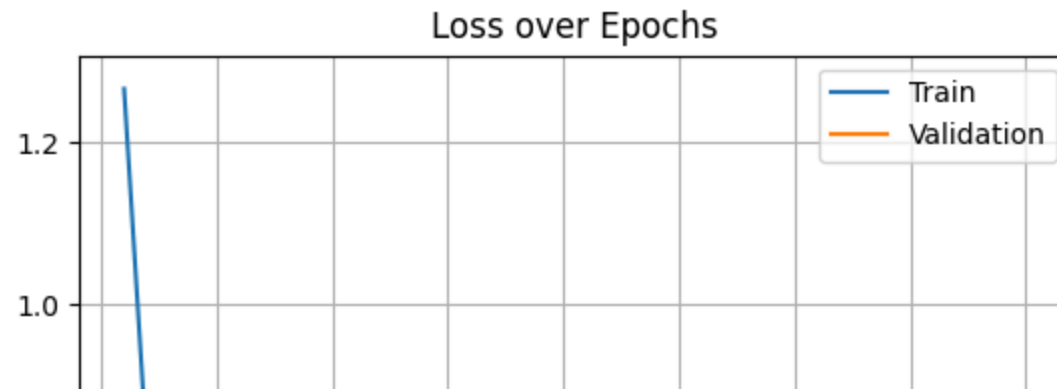
2.2.2 Accuracy over epoch

The accuracy plot shows a steady improvement in both training and validation accuracy throughout the training process. Validation accuracy slightly surpasses training accuracy in several epochs, which can be a sign of either strong generalization or variability in training data.



2.2.3 Loss over epoch

The loss plot supports these findings, with both training and validation loss decreasing gradually over time. The loss curves are relatively close to each other, with occasional fluctuations in the validation loss, which is typical in real-world training scenarios.



3. resnext50_32x4d

ResNeXt50_32x4d is a convolutional neural network that builds upon the ResNet architecture by adding cardinality, which means using multiple parallel paths within each block. The “50” refers to its depth (number of layers), while “32x4d” indicates it uses 32 groups with a width of 4.

3.1 *resnext50_32x4d* code


```

In [ ]: # Load model
base_model = torchvision.models.resnext50_32x4d(weights='DEFAULT')
model = CNN(base_model, num_classes)
model.to(device)

# Freeze everything
for param in model.parameters():
    param.requires_grad = False

# Unfreeze selected layers
print("✅ Unfreezing layers:", unfrozen_layer_names)
for name, module in base_model.named_children():
    if name in unfrozen_layer_names:
        for param in module.parameters():
            param.requires_grad = True

# Ensure classifier (fc) head is unfreezed from custom model
for param in model.fc.parameters():
    param.requires_grad = True

# Print trainable param count
trainable_params = list(filter(lambda p: p.requires_grad, model.parameters()))
print(f"🔍 Trainable parameters: {sum(p.numel() for p in trainable_params)}")

if not trainable_params:
    raise RuntimeError("❌ No trainable parameters found. Check layer unfreezing logic.")

# Optimizer and Loss
optimizer = torch.optim.Adam(trainable_params, lr=learning_rate)
criterion = nn.CrossEntropyLoss()

# Train
history = model.train_model(train_loader, valid_loader, optimizer, criterion, epochs=epochs, device=device)

# Save model
model_filename = f"resnext50_32x4d_model_batchsize{batch_size}_imgsize{img_size}_learningrate{learning_rate}_epoch{epoch}"
model_path = os.path.join(os.getcwd(), model_filename)
torch.save(model.state_dict(), model_path)

# === Save Plots === #
plot_folder = os.path.join("plots", os.path.splitext(model_filename)[0])
os.makedirs(plot_folder, exist_ok=True)

```

```
def plot_metric(metric_name, train_values, val_values):
    plt.figure()
    plt.plot(range(1, epochs + 1), train_values, label='Train')
    plt.plot(range(1, epochs + 1), val_values, label='Validation')
    plt.xlabel("Epoch")
    plt.ylabel(metric_name.capitalize())
    plt.title(f"{metric_name.capitalize()} over Epochs")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(plot_folder, f"{metric_name}.png"))
    plt.close()

# Plot and save all relevant metrics
plot_metric("loss", history["train_loss"], history["valid_loss"])
plot_metric("accuracy", history["train_accuracy"], history["valid_accuracy"])

# === Evaluate and Save Confusion Matrix === #
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in valid_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, preds = torch.max(outputs, 1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Compute confusion matrix
cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues', xticks_rotation='vertical')

# Save confusion matrix plot
plt.title("Confusion Matrix")
plt.savefig(os.path.join(plot_folder, "confusion_matrix.png"))
plt.close()

# Save classification report
report = classification_report(all_labels, all_preds, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.to_csv(os.path.join(plot_folder, "classification_report.csv"))
```



```
# Optional: also save raw confusion matrix  
cm_df = pd.DataFrame(cm)  
cm_df.to_csv(os.path.join(plot_folder, "confusion_matrix_raw.csv"), index=False)
```

✅ Unfreezing layers: ['layer4', 'fc']

🔍 Trainable parameters: 16647172

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:61: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.  
  with autocast():
```

Epoch 1/40 - Train Loss: 0.8144, Train Acc: 0.6771

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:86: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.  
  with autocast():
```

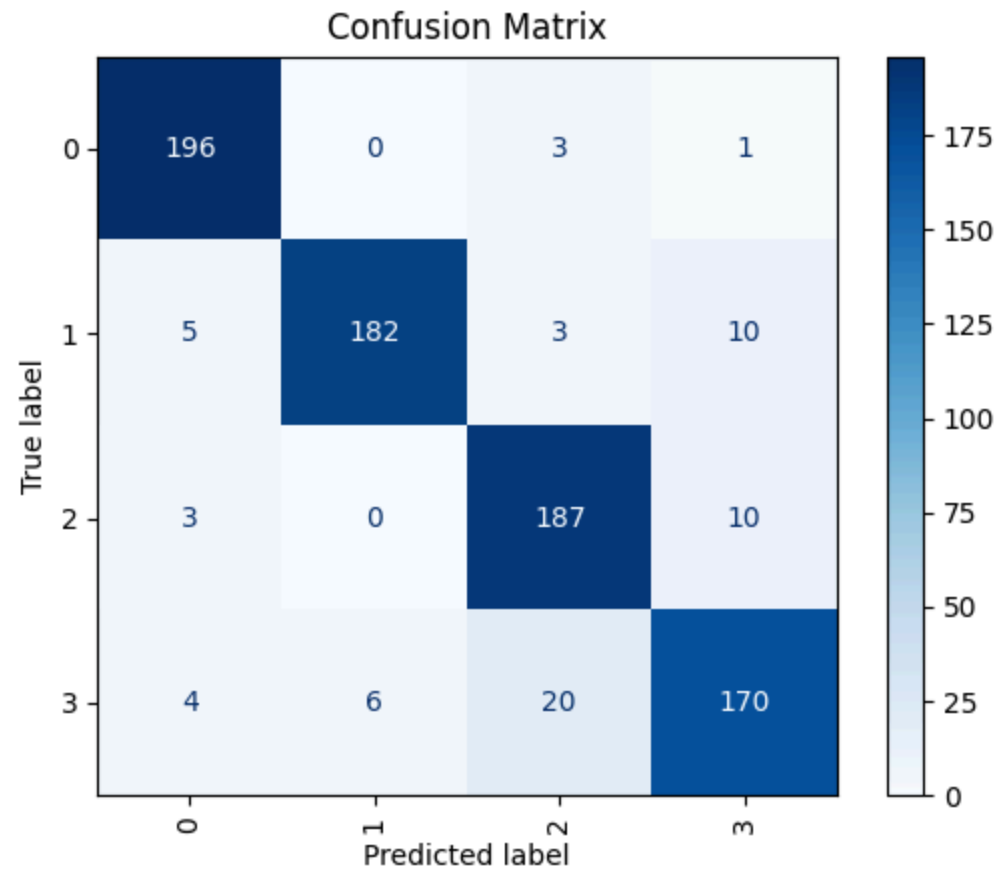
Epoch 1/40 - Val Loss: 0.7258, Val Acc: 0.7738
Epoch 2/40 - Train Loss: 0.5789, Train Acc: 0.7713
Epoch 2/40 - Val Loss: 1.1044, Val Acc: 0.6750
Epoch 3/40 - Train Loss: 0.4881, Train Acc: 0.8114
Epoch 3/40 - Val Loss: 0.5688, Val Acc: 0.8325
Epoch 4/40 - Train Loss: 0.4472, Train Acc: 0.8290
Epoch 4/40 - Val Loss: 1.1896, Val Acc: 0.8187
Epoch 5/40 - Train Loss: 0.4037, Train Acc: 0.8437
Epoch 5/40 - Val Loss: 0.4361, Val Acc: 0.8625
Epoch 6/40 - Train Loss: 0.3760, Train Acc: 0.8531
Epoch 6/40 - Val Loss: 1.0110, Val Acc: 0.8462
Epoch 7/40 - Train Loss: 0.3465, Train Acc: 0.8631
Epoch 7/40 - Val Loss: 1.6980, Val Acc: 0.8250
Epoch 8/40 - Train Loss: 0.3362, Train Acc: 0.8674
Epoch 8/40 - Val Loss: 0.5272, Val Acc: 0.8712
Epoch 9/40 - Train Loss: 0.3343, Train Acc: 0.8725
Epoch 9/40 - Val Loss: 0.6547, Val Acc: 0.8788
Epoch 10/40 - Train Loss: 0.3136, Train Acc: 0.8841
Epoch 10/40 - Val Loss: 2.8245, Val Acc: 0.8475
Epoch 11/40 - Train Loss: 0.2846, Train Acc: 0.8958
Epoch 11/40 - Val Loss: 0.4244, Val Acc: 0.8750
Epoch 12/40 - Train Loss: 0.2682, Train Acc: 0.8972
Epoch 12/40 - Val Loss: 1.3242, Val Acc: 0.8725
Epoch 13/40 - Train Loss: 0.2660, Train Acc: 0.8988
Epoch 13/40 - Val Loss: 2.3494, Val Acc: 0.8988
Epoch 14/40 - Train Loss: 0.2775, Train Acc: 0.8922
Epoch 14/40 - Val Loss: 0.7156, Val Acc: 0.8200
Epoch 15/40 - Train Loss: 0.2589, Train Acc: 0.9025
Epoch 15/40 - Val Loss: 1.7118, Val Acc: 0.7812
Epoch 16/40 - Train Loss: 0.2638, Train Acc: 0.9022
Epoch 16/40 - Val Loss: 0.6265, Val Acc: 0.8862
Epoch 17/40 - Train Loss: 0.2393, Train Acc: 0.9045
Epoch 17/40 - Val Loss: 1.0570, Val Acc: 0.8812
Epoch 18/40 - Train Loss: 0.2346, Train Acc: 0.9129
Epoch 18/40 - Val Loss: 5.1798, Val Acc: 0.8488
Epoch 19/40 - Train Loss: 0.2027, Train Acc: 0.9222
Epoch 19/40 - Val Loss: 1.0136, Val Acc: 0.8550
Epoch 20/40 - Train Loss: 0.2291, Train Acc: 0.9122
Epoch 20/40 - Val Loss: nan, Val Acc: 0.8875
Epoch 21/40 - Train Loss: 0.2092, Train Acc: 0.9192
Epoch 21/40 - Val Loss: 9.3646, Val Acc: 0.8675
Epoch 22/40 - Train Loss: 0.2280, Train Acc: 0.9179
Epoch 22/40 - Val Loss: 4.8681, Val Acc: 0.8975

Epoch 23/40 - Train Loss: 0.1926, Train Acc: 0.9272
Epoch 23/40 - Val Loss: 6.3737, Val Acc: 0.9038
Epoch 24/40 - Train Loss: 0.1887, Train Acc: 0.9262
Epoch 24/40 - Val Loss: 0.6190, Val Acc: 0.8850
Epoch 25/40 - Train Loss: 0.1727, Train Acc: 0.9379
Epoch 25/40 - Val Loss: 3.0740, Val Acc: 0.8888
Epoch 26/40 - Train Loss: 0.1952, Train Acc: 0.9239
Epoch 26/40 - Val Loss: 18.6703, Val Acc: 0.8488
Epoch 27/40 - Train Loss: 0.1876, Train Acc: 0.9302
Epoch 27/40 - Val Loss: 5.4582, Val Acc: 0.8500
Epoch 28/40 - Train Loss: 0.1734, Train Acc: 0.9399
Epoch 28/40 - Val Loss: 5.0791, Val Acc: 0.8912
Epoch 29/40 - Train Loss: 0.1679, Train Acc: 0.9406
Epoch 29/40 - Val Loss: 6.0062, Val Acc: 0.8862
Epoch 30/40 - Train Loss: 0.1561, Train Acc: 0.9419
Epoch 30/40 - Val Loss: 2.8912, Val Acc: 0.8712
Epoch 31/40 - Train Loss: 0.1604, Train Acc: 0.9399
Epoch 31/40 - Val Loss: nan, Val Acc: 0.8962
Epoch 32/40 - Train Loss: 0.1650, Train Acc: 0.9386
Epoch 32/40 - Val Loss: 3.1603, Val Acc: 0.9050
Epoch 33/40 - Train Loss: 0.1465, Train Acc: 0.9472
Epoch 33/40 - Val Loss: 2.2666, Val Acc: 0.9200
Epoch 34/40 - Train Loss: 0.1534, Train Acc: 0.9416
Epoch 34/40 - Val Loss: 4.0241, Val Acc: 0.9125
Epoch 35/40 - Train Loss: 0.1767, Train Acc: 0.9396
Epoch 35/40 - Val Loss: 1.3237, Val Acc: 0.8725
Epoch 36/40 - Train Loss: 0.1434, Train Acc: 0.9496
Epoch 36/40 - Val Loss: nan, Val Acc: 0.8488
Epoch 37/40 - Train Loss: 0.1501, Train Acc: 0.9449
Epoch 37/40 - Val Loss: 4.7515, Val Acc: 0.8712
Epoch 38/40 - Train Loss: 0.1481, Train Acc: 0.9456
Epoch 38/40 - Val Loss: 5.9304, Val Acc: 0.8612
Epoch 39/40 - Train Loss: 0.1731, Train Acc: 0.9402
Epoch 39/40 - Val Loss: 5.0000, Val Acc: 0.8750
Epoch 40/40 - Train Loss: 0.1712, Train Acc: 0.9352
Epoch 40/40 - Val Loss: 1.4097, Val Acc: 0.8988

3.2 resnext50_32x4d Results

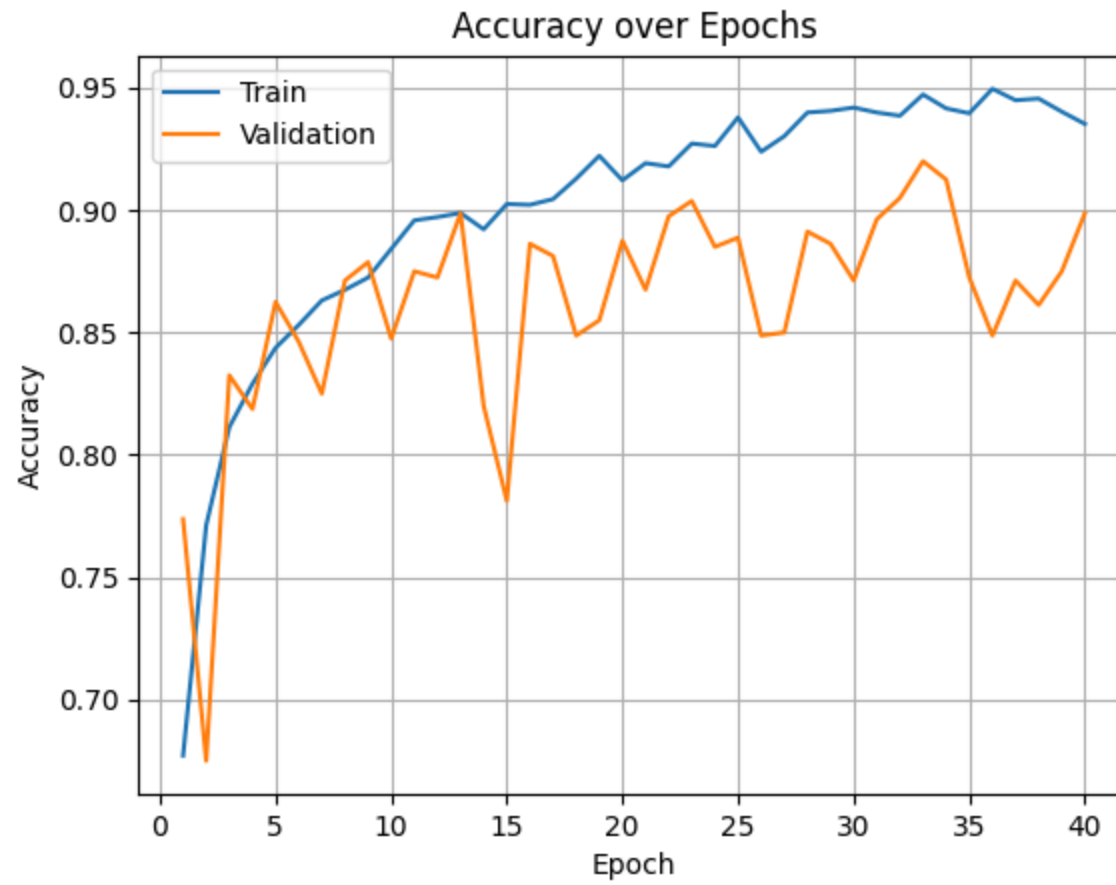
3.2.1 Confusion Matrix:

The confusion matrix demonstrates strong overall classification performance. Classes 0, 1, and 2 are classified with high accuracy, with very few misclassifications. Class 3, while still well-classified, shows some confusion—especially being misclassified as Class 2



3.2.2 Accuracy over epoch

The accuracy plot indicates effective learning, with training accuracy steadily improving and reaching approximately 95%. Validation accuracy follows a similar trend and fluctuates between 85% and 91%, which suggests good generalization



3.2.3 Loss over epoch

The loss plot presents an anomaly. While training loss remains low and stable throughout the training process, validation loss shows significant volatility, with occasional extreme spikes. These sharp increases likely reflect instability in the validation process—possibly due to noisy or inconsistent batches, outliers



4. resnet50

ResNet50 is a widely used convolutional neural network known for its deep architecture and efficient performance. It consists of 50 layers and introduces residual connections, which help prevent vanishing gradients and allow the model to learn deeper representations effectively.

4.1 resnet50 code


```

In [ ]: # Load model
base_model = torchvision.models.resnet50(weights='DEFAULT')
model = CNN(base_model, num_classes)
model.to(device)

# Freeze everything
for param in model.parameters():
    param.requires_grad = False

# Unfreeze selected layers
print("✅ Unfreezing layers:", unfrozen_layer_names)
for name, module in base_model.named_children():
    if name in unfrozen_layer_names:
        for param in module.parameters():
            param.requires_grad = True

# Ensure classifier (fc) head is unfreezed from custom model
for param in model.fc.parameters():
    param.requires_grad = True

# Print trainable param count
trainable_params = list(filter(lambda p: p.requires_grad, model.parameters()))
print(f"🔍 Trainable parameters: {sum(p.numel() for p in trainable_params)}")

if not trainable_params:
    raise RuntimeError("❌ No trainable parameters found. Check layer unfreezing logic.")

# Optimizer and Loss
optimizer = torch.optim.Adam(trainable_params, lr=learning_rate)
criterion = nn.CrossEntropyLoss()

# Train
history = model.train_model(train_loader, valid_loader, optimizer, criterion, epochs=epochs, device=device)

# Save model
model_filename = f"resnet50_model_batchsize{batch_size}_imgsize{img_size}_learningrate{learning_rate}_epoch{epoch}"
model_path = os.path.join(os.getcwd(), model_filename)
torch.save(model.state_dict(), model_path)

# === Save Plots === #
plot_folder = os.path.join("plots", os.path.splitext(model_filename)[0])
os.makedirs(plot_folder, exist_ok=True)

```



```
def plot_metric(metric_name, train_values, val_values):
    plt.figure()
    plt.plot(range(1, epochs + 1), train_values, label='Train')
    plt.plot(range(1, epochs + 1), val_values, label='Validation')
    plt.xlabel("Epoch")
    plt.ylabel(metric_name.capitalize())
    plt.title(f"{metric_name.capitalize()} over Epochs")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(plot_folder, f"{metric_name}.png"))
    plt.close()

# Plot and save all relevant metrics
plot_metric("loss", history["train_loss"], history["valid_loss"])
plot_metric("accuracy", history["train_accuracy"], history["valid_accuracy"])

# === Evaluate and Save Confusion Matrix === #
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in valid_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, preds = torch.max(outputs, 1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Compute confusion matrix
cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues', xticks_rotation='vertical')

# Save confusion matrix plot
plt.title("Confusion Matrix")
plt.savefig(os.path.join(plot_folder, "confusion_matrix.png"))
plt.close()

# Save classification report
report = classification_report(all_labels, all_preds, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.to_csv(os.path.join(plot_folder, "classification_report.csv"))
```

```
# Optional: also save raw confusion matrix
cm_df = pd.DataFrame(cm)
cm_df.to_csv(os.path.join(plot_folder, "confusion_matrix_raw.csv"), index=False)
```

✅ Unfreezing layers: ['layer4', 'fc']

🔍 Trainable parameters: 17067012

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:43: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
```

```
    scaler = GradScaler()
```

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:61: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with autocast():
```

Epoch 1/40 - Train Loss: 0.7660, Train Acc: 0.6878

```
c:\Users\guill\Documents\master\ImagenAdne\PracticaImagen\src\cnn.py:86: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
    with autocast():
```

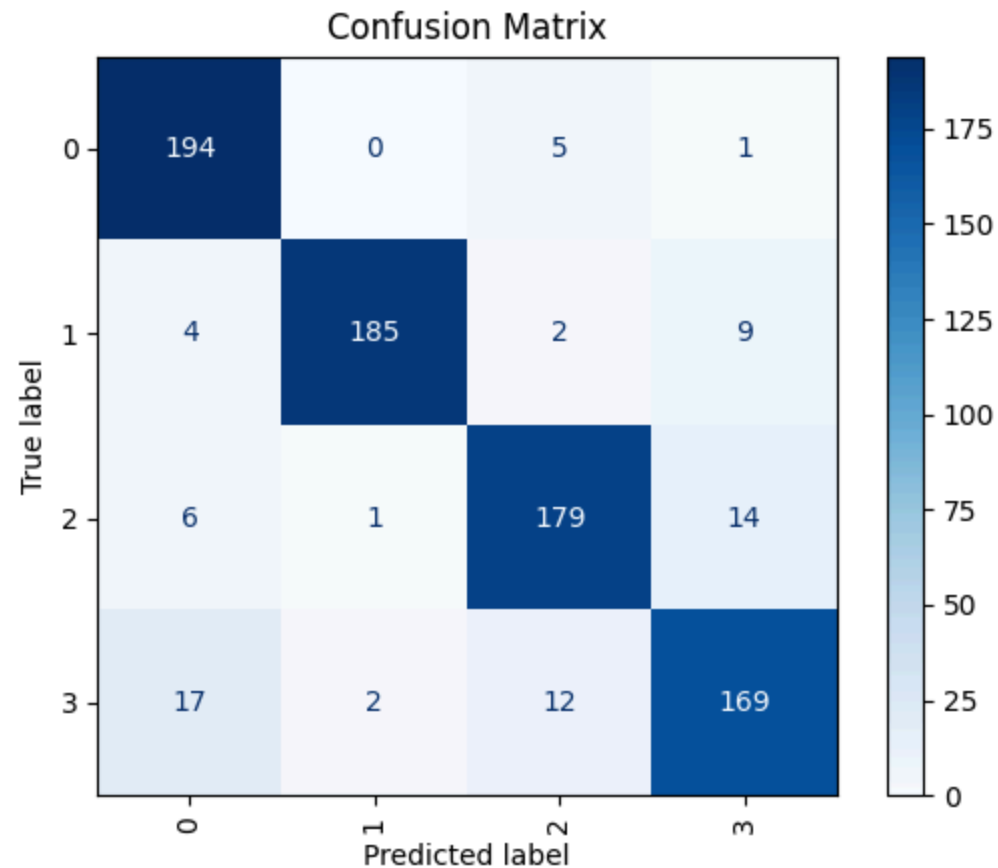
Epoch 1/40 - Val Loss: 0.6314, Val Acc: 0.7625
Epoch 2/40 - Train Loss: 0.5634, Train Acc: 0.7790
Epoch 2/40 - Val Loss: 0.5380, Val Acc: 0.8025
Epoch 3/40 - Train Loss: 0.4723, Train Acc: 0.8264
Epoch 3/40 - Val Loss: 0.5349, Val Acc: 0.8187
Epoch 4/40 - Train Loss: 0.4595, Train Acc: 0.8160
Epoch 4/40 - Val Loss: 0.7725, Val Acc: 0.8350
Epoch 5/40 - Train Loss: 0.4069, Train Acc: 0.8411
Epoch 5/40 - Val Loss: 0.4885, Val Acc: 0.8325
Epoch 6/40 - Train Loss: 0.3827, Train Acc: 0.8574
Epoch 6/40 - Val Loss: 0.4425, Val Acc: 0.8275
Epoch 7/40 - Train Loss: 0.3788, Train Acc: 0.8521
Epoch 7/40 - Val Loss: 0.5644, Val Acc: 0.8325
Epoch 8/40 - Train Loss: 0.3785, Train Acc: 0.8504
Epoch 8/40 - Val Loss: 0.3615, Val Acc: 0.8750
Epoch 9/40 - Train Loss: 0.3148, Train Acc: 0.8765
Epoch 9/40 - Val Loss: 0.6786, Val Acc: 0.8588
Epoch 10/40 - Train Loss: 0.3332, Train Acc: 0.8745
Epoch 10/40 - Val Loss: 1.5635, Val Acc: 0.7488
Epoch 11/40 - Train Loss: 0.3066, Train Acc: 0.8808
Epoch 11/40 - Val Loss: 0.4204, Val Acc: 0.8775
Epoch 12/40 - Train Loss: 0.2955, Train Acc: 0.8888
Epoch 12/40 - Val Loss: 0.4065, Val Acc: 0.8650
Epoch 13/40 - Train Loss: 0.2963, Train Acc: 0.8798
Epoch 13/40 - Val Loss: 0.6060, Val Acc: 0.8888
Epoch 14/40 - Train Loss: 0.2775, Train Acc: 0.8942
Epoch 14/40 - Val Loss: 0.7873, Val Acc: 0.8662
Epoch 15/40 - Train Loss: 0.2708, Train Acc: 0.8962
Epoch 15/40 - Val Loss: 0.6284, Val Acc: 0.8612
Epoch 16/40 - Train Loss: 0.2676, Train Acc: 0.8955
Epoch 16/40 - Val Loss: 0.4381, Val Acc: 0.8700
Epoch 17/40 - Train Loss: 0.2498, Train Acc: 0.9055
Epoch 17/40 - Val Loss: 0.5704, Val Acc: 0.8688
Epoch 18/40 - Train Loss: 0.2866, Train Acc: 0.8868
Epoch 18/40 - Val Loss: 0.6524, Val Acc: 0.8250
Epoch 19/40 - Train Loss: 0.2473, Train Acc: 0.9052
Epoch 19/40 - Val Loss: 0.5020, Val Acc: 0.8650
Epoch 20/40 - Train Loss: 0.2434, Train Acc: 0.9082
Epoch 20/40 - Val Loss: 0.6955, Val Acc: 0.8788
Epoch 21/40 - Train Loss: 0.2428, Train Acc: 0.9075
Epoch 21/40 - Val Loss: 0.9556, Val Acc: 0.8387
Epoch 22/40 - Train Loss: 0.2203, Train Acc: 0.9179
Epoch 22/40 - Val Loss: 0.8357, Val Acc: 0.8225

Epoch 23/40 - Train Loss: 0.2496, Train Acc: 0.9052
Epoch 23/40 - Val Loss: 0.8862, Val Acc: 0.8850
Epoch 24/40 - Train Loss: 0.2225, Train Acc: 0.9149
Epoch 24/40 - Val Loss: 0.5432, Val Acc: 0.9025
Epoch 25/40 - Train Loss: 0.2229, Train Acc: 0.9142
Epoch 25/40 - Val Loss: 0.8800, Val Acc: 0.8712
Epoch 26/40 - Train Loss: 0.2016, Train Acc: 0.9209
Epoch 26/40 - Val Loss: 0.8008, Val Acc: 0.9113
Epoch 27/40 - Train Loss: 0.1987, Train Acc: 0.9252
Epoch 27/40 - Val Loss: 0.8846, Val Acc: 0.8875
Epoch 28/40 - Train Loss: 0.2213, Train Acc: 0.9192
Epoch 28/40 - Val Loss: 0.9764, Val Acc: 0.8688
Epoch 29/40 - Train Loss: 0.1915, Train Acc: 0.9285
Epoch 29/40 - Val Loss: 0.4420, Val Acc: 0.8738
Epoch 30/40 - Train Loss: 0.2147, Train Acc: 0.9222
Epoch 30/40 - Val Loss: 0.7011, Val Acc: 0.8462
Epoch 31/40 - Train Loss: 0.2014, Train Acc: 0.9242
Epoch 31/40 - Val Loss: 0.4276, Val Acc: 0.8862
Epoch 32/40 - Train Loss: 0.1902, Train Acc: 0.9242
Epoch 32/40 - Val Loss: 1.7483, Val Acc: 0.7975
Epoch 33/40 - Train Loss: 0.1971, Train Acc: 0.9275
Epoch 33/40 - Val Loss: 0.4828, Val Acc: 0.8850
Epoch 34/40 - Train Loss: 0.1900, Train Acc: 0.9245
Epoch 34/40 - Val Loss: 0.3540, Val Acc: 0.8850
Epoch 35/40 - Train Loss: 0.2076, Train Acc: 0.9215
Epoch 35/40 - Val Loss: 1.3066, Val Acc: 0.8337
Epoch 36/40 - Train Loss: 0.1784, Train Acc: 0.9332
Epoch 36/40 - Val Loss: 0.6228, Val Acc: 0.8862
Epoch 37/40 - Train Loss: 0.1843, Train Acc: 0.9359
Epoch 37/40 - Val Loss: 2.1603, Val Acc: 0.8337
Epoch 38/40 - Train Loss: 0.1846, Train Acc: 0.9306
Epoch 38/40 - Val Loss: 0.4710, Val Acc: 0.8850
Epoch 39/40 - Train Loss: 0.1751, Train Acc: 0.9326
Epoch 39/40 - Val Loss: 1.0288, Val Acc: 0.8450
Epoch 40/40 - Train Loss: 0.1649, Train Acc: 0.9356
Epoch 40/40 - Val Loss: 1.6938, Val Acc: 0.8938

4.2 resnet50 Results

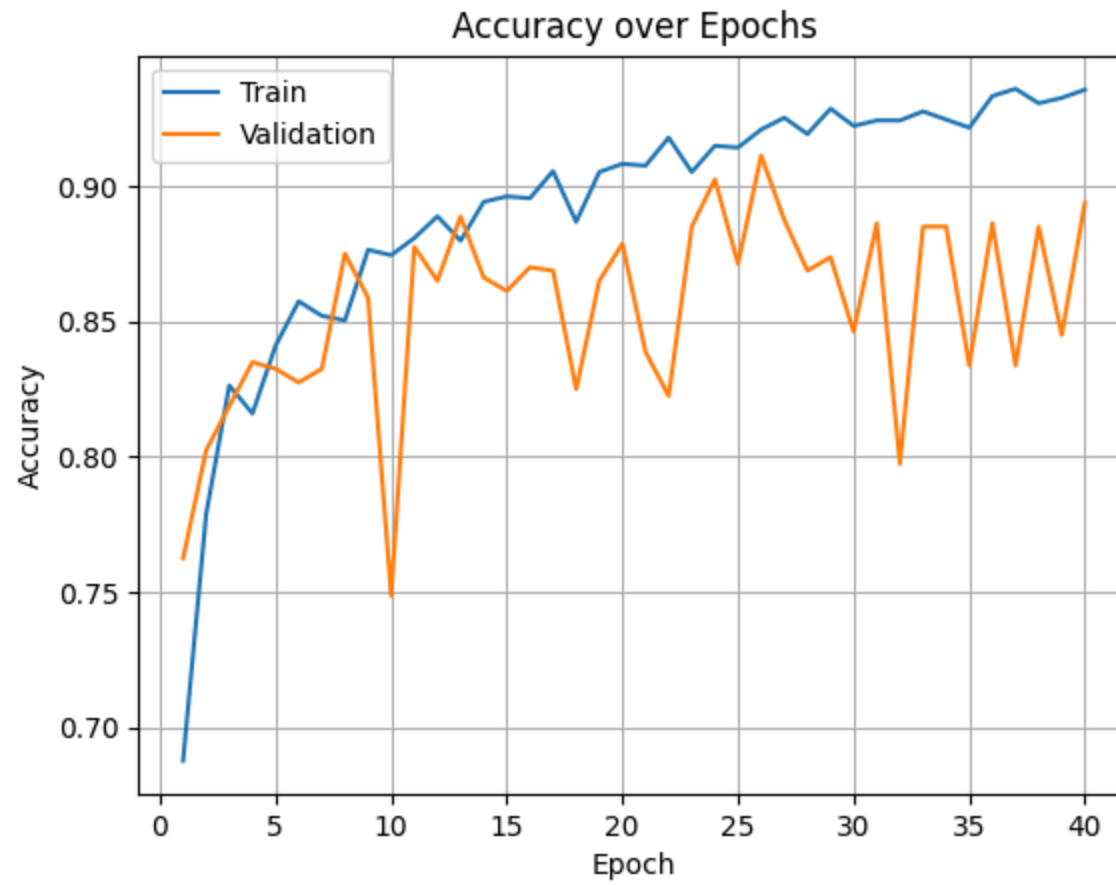
4.2.1 Confusion Matrix:

The confusion matrix indicates strong classification performance across all four classes. Class 0 and Class 1 are predicted with high accuracy, showing minimal misclassification. Class 2 is also handled well, although it shows some overlap with Class 3. Class 3, while still accurately classified in most cases



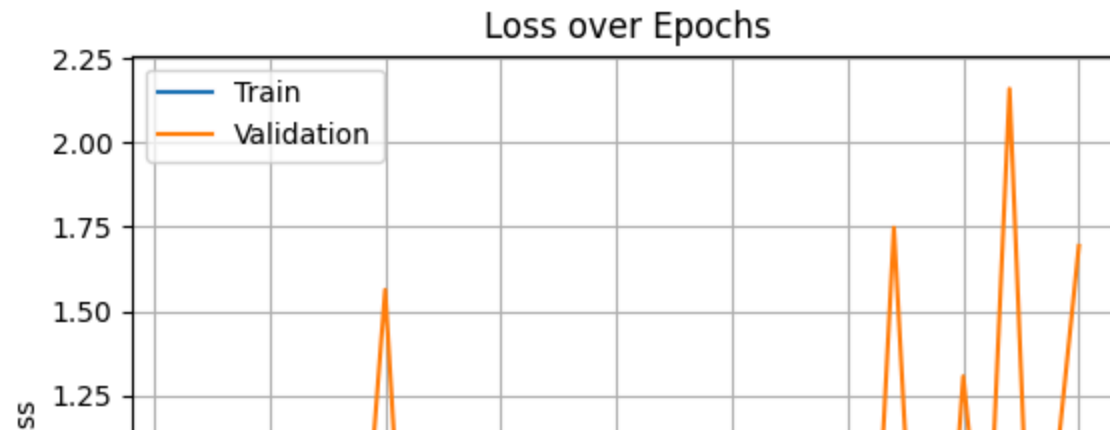
4.2.2 Accuracy over epoch

The accuracy over epochs plot shows steady progress in both training and validation accuracy. Training accuracy gradually increases, surpassing 92% by the end of training. Validation accuracy follows a similar trend, although it shows greater variability, oscillating between approximately 85% and 91%. This variability suggests the model generalizes fairly well but may be sensitive to certain validation samples.



4.2.3 Loss over epoch

The loss plot reveals notable instability in the validation process. While the training loss consistently decreases and remains low, validation loss fluctuates significantly, with several sharp spikes throughout the training period. These spikes indicate that, despite good accuracy, the model's predictions on the validation set are sometimes inconsistent or overly confident,



Conclusion

All four models performed well in classifying eye diseases, with validation accuracies consistently above 85%. **ResNeXt101_64x4d** showed the best overall performance, combining high accuracy with stable training. **ResNeXt50_32x4d** followed closely, offering a good trade-off between performance and efficiency. **RegNetY-128GF** achieved strong results but showed some inconsistency, while **ResNet50**, though simpler, delivered competitive accuracy with less stable validation loss. Overall, **ResNeXt101_64x4d** proved to be the most reliable choice.

Comparing the results with the ML and DL models, it is worth noting the great surprise of the good results of ML, reaching 90% accuracy and surpassing commercial models. However, the best models in each of the three phases obtain very similar results, which indicates that it is not necessary to make the problem very complex, but rather a good feature extraction.